SWT 22022 – Practical for Internet Application Development

Department of Information and Communication Technology

Faculty of Technology

Lab sheet :19
Reg. Number: SEU/IS/20/ICT/084
Academic Year :2020/2021
Date: 2024.10.30
Practical No :20

Title:

- React Routes and API

Aims:

- Creating a dynamic list
- Using context to share data between components
- Building a simple routing system
- API and Axios

Tasks:

- Use **useState** to manage list
- Use **Context API** to pass data between components without props drilling.
- Use **React Router** for client-side routing
- Use **Axios** to fetch data from an API

Activities :

1. Use **useState** to manage list

```
import { useState } from 'react';
const DynamicList = () => {
      const [items, setItems] = useState([]);
      const addItem = () => setItems([...items, `Item ${items.length +
1}`]);
      return (
            <div>
                  <button onClick={addItem}>Add Item</button>
                  <ul>{items.map((item, index) => <li
            key={index}>{item}</li>)}
                  </ul>
            </div>
            );
};
export default DynamicList;
```

2. Use Context API to pass data between components without props drilling.

a. ContextProvider.jsx

```
import { createContext, useState } from 'react';
export const MyContext = createContext();
const ContextProvider = ({ children }) => {
    const [sharedData, setSharedData] = useState("Initial Data");
    return (
```

```
        <MyContext.Provider value={{ sharedData, setSharedData
}}>{children}</MyContext.Provider>
    );
};
export default ContextProvider;
```

b. ComponentNew.jsx

```
import { useContext } from 'react';
import { MyContext } from './ContextProvider';
const ComponentNew = () => {
    const { sharedData, setSharedData } = useContext(MyContext);
    return (
        <div>
            <p>{sharedData}</p>
            <button onClick={() => setSharedData("Updated
Data")}>Update</button>
        </div>
    );
};
export default ComponentNew;
```

3. Use React Router for client-side routing

Install React Router :

npm install react-router-dom

a. Home.jsx

```
const Home = () => {
    return <h2>Home</h2>;
    };
export default Home;
```

b. About.jsx

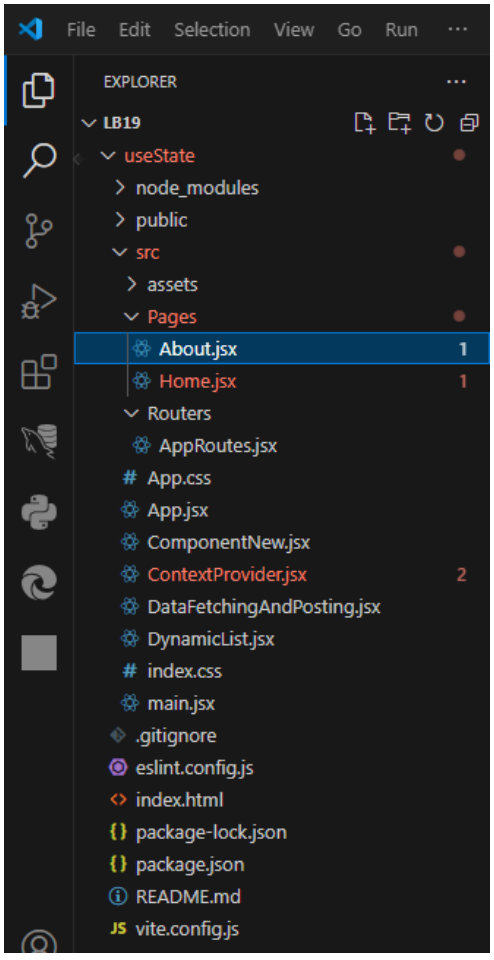
```
const About = () => {
    return <h2>About</h2>;
    };
export default About;
```

c. AppRoutes.jsx

```
import { Routes, Route } from 'react-router-dom';
import Home from './Home';
import About from './About';
const AppRoutes = () => {
    return (
        <Routes>
            <Route path="/" element={<Home />} />
            <Route path="/about" element={<About />} />
            </Routes>
        );
};
export default AppRoutes;
```

d. App.jsx

```
import { BrowserRouter as Router, Link } from 'react-router-dom';
import AppRoutes from './AppRoutes';
const App = () => (
    <Router>
        <nav>
            <Link to="/">Home</Link>
            <Link to="/about">About</Link>
        </nav>
        <AppRoutes />
    </Router>
);
export default App;
```



About.jsx

```
const About = () => {
    return (
        <div>
                <h2>This is About Page Content</h2>
                <h1>What is Lorem Ipsum?</h1>
                <p>Lorem Ipsum is simply dummy text of the printing and
typesetting industry. Lorem Ipsum has been the industry's standard dummy text
ever since the 1500s, when an unknown printer took a galley of type and scrambled
it to make a type specimen book. It has survived not only five centuries, but
also the leap into electronic typesetting, remaining essentially unchanged. It
was popularised in the 1960s with the release of Letraset sheets containing Lorem
Ipsum passages, and more recently with desktop publishing software like Aldus
PageMaker including versions of Lorem Ipsum.</p>
        </div>
    );


};
export default About;
```

Home.jsx

```
const About = () => {
    return (
        <div>
                <h2>This is About Page Content</h2>
                <h1>What is Lorem Ipsum?</h1>
                <p>Lorem Ipsum is simply dummy text of the printing and
typesetting industry. Lorem Ipsum has been the industry's standard dummy text
ever since the 1500s, when an unknown printer took a galley of type and scrambled
it to make a type specimen book. It has survived not only five centuries, but
also the leap into electronic typesetting, remaining essentially unchanged. It
was popularised in the 1960s with the release of Letraset sheets containing Lorem
Ipsum passages, and more recently with desktop publishing software like Aldus
PageMaker including versions of Lorem Ipsum.</p>
        </div>
    );


};
export default About;
```

DynamicList.jsx

```
import { useState } from 'react';

const DynamicList = () => {
  const [items, setItems] = useState([]);
  const addItem = () => setItems([...items, `Item ${items.length + 1}`]);

  return (
    <div>
      <button onClick={addItem}>Add Item</button>
      <ul>{items.map((item, index) => <li key={index}>{item}</li>)}</ul>
    </div>
  );
};

export default DynamicList;
```

ContextProvider.jsx

```
import { useState } from 'react';

const DynamicList = () => {
  const [items, setItems] = useState([]);
  const addItem = () => setItems([...items, `Item ${items.length + 1}`]);

  return (
    <div>
      <button onClick={addItem}>Add Item</button>
      <ul>{items.map((item, index) => <li key={index}>{item}</li>)}</ul>
    </div>
  );
};

export default DynamicList;
```

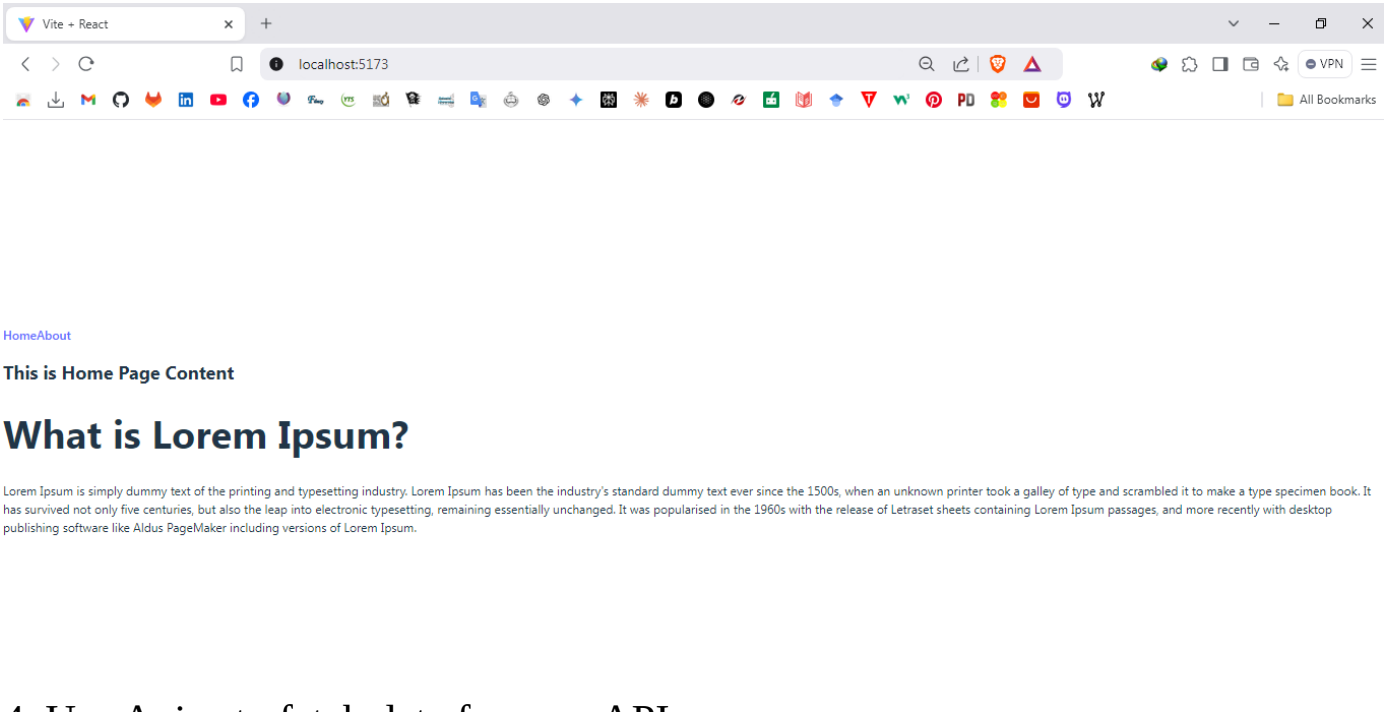
ComponentNew.jsx

```
import { useContext } from 'react';
import { MyContext } from './ContextProvider';
const ComponentNew = () => {
    const { sharedData, setSharedData } = useContext(MyContext);
    return (
        <div>
            <p>{sharedData}</p>
            <button onClick={() => setSharedData("Updated
Data")}>Update</button>
        </div>
    );
```

```
};
export default ComponentNew;
```

AppRoutes.jsx

```
import { Routes, Route } from 'react-router-dom';
import Home from '../Pages/Home';
import About from '../Pages/About';
const AppRoutes = () => {
    return (
        <Routes>
            <Route path="/" element={<Home />} />
            <Route path="/about" element={<About />} />
        </Routes>
    );
};
export default AppRoutes;
```



4. Use Axios to fetch data from an API

Install Axios :

npm install axios

a. DataFetchingAndPosting.jsx

```
import { useEffect, useState } from 'react';
import axios from 'axios';
const DataFetchingAndPosting = () => {
    const [data, setData] = useState([]);
```

```
    const [newPost, setNewPost] = useState({
        title: '',
        body: '',
        userId: 1
    });
    // GET request to fetch data
    useEffect(() => {
        axios.get('https://jsonplaceholder.typicode.com/posts').then(response =>
setData(response.data)).catch(error => console.error(error));
    }, []);
    // POST request to add new data
    const handlePost = () => {
        axios.post('https://jsonplaceholder.typicode.com/posts',newPost).then(re
sponse => {
            console.log('New Post Added:', response.data);
            setData([...data, response.data]);
        }).catch(error => console.error(error));
    };
    return (
        <div>
            <h3>Data Fetching (GET Request)</h3>
            <ul>{data.map(post => <li key={post.id}>{post.title}</li>)}</ul>
            <h3>Add a New Post (POST Request)</h3>
            <input type="text" placeholder="Title" value={newPost.title}
onChange={e => setNewPost({ ...newPost, title: e.target.value })}/>
            <input type="text" placeholder="Body" value={newPost.body}
onChange={e => setNewPost({ ...newPost, body:e.target.value })}/>
            <button onClick={handlePost}>Add Post</button>
        </div>
    );
};
export default DataFetchingAndPosting;
```

b. App.jsx

```
import DataFetchingAndPosting from './DataFetchingAndPosting';
function App() {
    return (
        <>
            <DataFetchingAndPosting />
        </>
    );
}
export default App;
```

DataFetchingAndPosting.jsx

```
import { useEffect, useState } from 'react';
import axios from 'axios';
const DataFetchingAndPosting = () => {
    const [data, setData] = useState([]);
    const [newPost, setNewPost] = useState({
        title: '',
        body: '',
        userId: 1
    });
    // GET request to fetch data
    useEffect(() => {
        axios.get('https://jsonplaceholder.typicode.com/posts').then(respons
e => setData(response.data)).catch(error => console.error(error));
    }, []);
    // POST request to add new data
    const handlePost = () => {
        axios.post('https://jsonplaceholder.typicode.com/posts',newPost).the
n(response => {
            console.log('New Post Added:', response.data);
            setData([...data, response.data]);
        }).catch(error => console.error(error));
    };
    return (
        <div>
            <h3>Data Fetching (GET Request)</h3>
            <ul>
                {data.map(post => <li key={post.id}>{post.title}</li>)}
            </ul>
            <h3>Add a New Post (POST Request)</h3>
            <input type="text" placeholder="Title" value={newPost.title}
onChange={e => setNewPost({ ...newPost, title: e.target.value })}/>
            <input type="text" placeholder="Body" value={newPost.body}
onChange={e => setNewPost({ ...newPost, body: e.target.value })}/>
            <button onClick={handlePost}>Add Post</button>
        </div>
    );
};
export default DataFetchingAndPosting;
```

App.jsx

```
import DataFetchingAndPosting from './DataFetchingAndPosting';
function App() {
    return (
        <>
            <DataFetchingAndPosting />
        </>
    );
}
export default App;
```

Discussion:

- In this lab session on React Routes and API integration, we explored fundamental concepts to build dynamic and interactive web applications. The session focused on creating a dynamic list using the `useState` hook to manage component-level state effectively. We utilized the Context API to share data seamlessly between components, avoiding the complexity of props drilling. This approach not only simplifies state management but also enhances the maintainability of the code. React Router was introduced to set up client-side routing, enabling smooth navigation between multiple pages like "Home" and "About." Additionally, we learned to fetch data from an API using Axios, demonstrating both GET requests for data retrieval and POST requests to add new entries dynamically. Overall, the session provided a hands-on understanding of core React functionalities and their application in building scalable web applications.